

กลุ่มที่ 5
การออกแบบและวิเคราะห์อัล
กอริทึมด้วย **STACK**

ขั้นตอนการทำงานฉุกเฉิน: ลำดับเหตุการณ์และเครื่องสถานะ

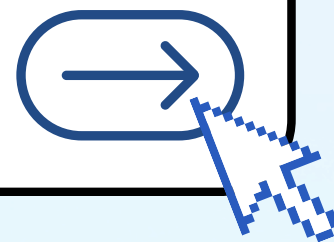
สมาชิก

นาย นายณัฐนัน เองฉ้วน 68122250001

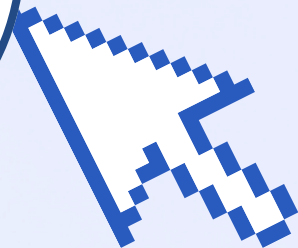
นาย ธนกร ชุมเดชา 68122250029

นาย พุฒิชัย โพธิ์ขำ 68122250094

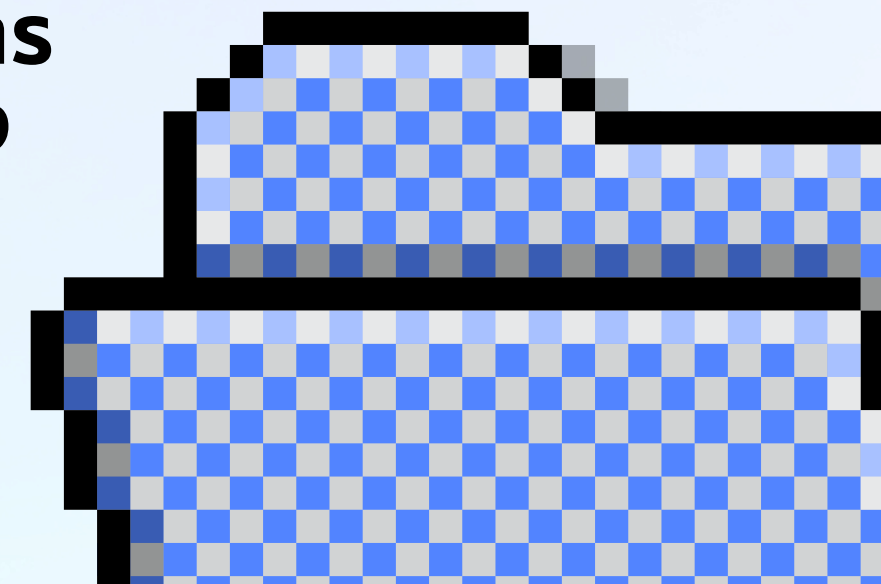
นาย ตะวัน นามลนนวน 67122250094

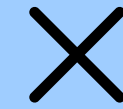


Case Story



- ศูนย์รับแจ้งเหตุฉุกเฉินบันทึกลำดับเหตุการณ์ตั้งแต่ รับแจ้งเหตุ มอบหมายทีม ส่งรถฉุกเฉิน เดินทางถึงจุดเกิดเหตุ และปิดเคส โดยต้องดำเนินการตามลำดับที่กำหนด
- เมื่อมี Action ใหม่ ระบบต้องตรวจสอบลำดับก่อนเพิ่มเข้า Event Stack เพื่อป้องกันการทำงานผิดพลาด เช่น ไม่สามารถส่งรถฉุกเฉินก่อนมอบหมายทีม หรือปิดเคสก่อนถึงจุดเกิดเหตุ
- ระบบต้องรองรับ Undo และ Redo โดยเมื่อ Undo จะนำ Action ล่าสุดออกจาก Event Stack ไปเก็บใน Redo Stack และเมื่อ Redo จะนำ Action กลับมา หากมีการเพิ่ม Action ใหม่หลัง Undo ต้องล้าง Redo Stack เพื่อให้ลำดับเหตุการณ์ถูกต้อง





Input, Output, Constraints

Input

- Action
- Undo
- Redo

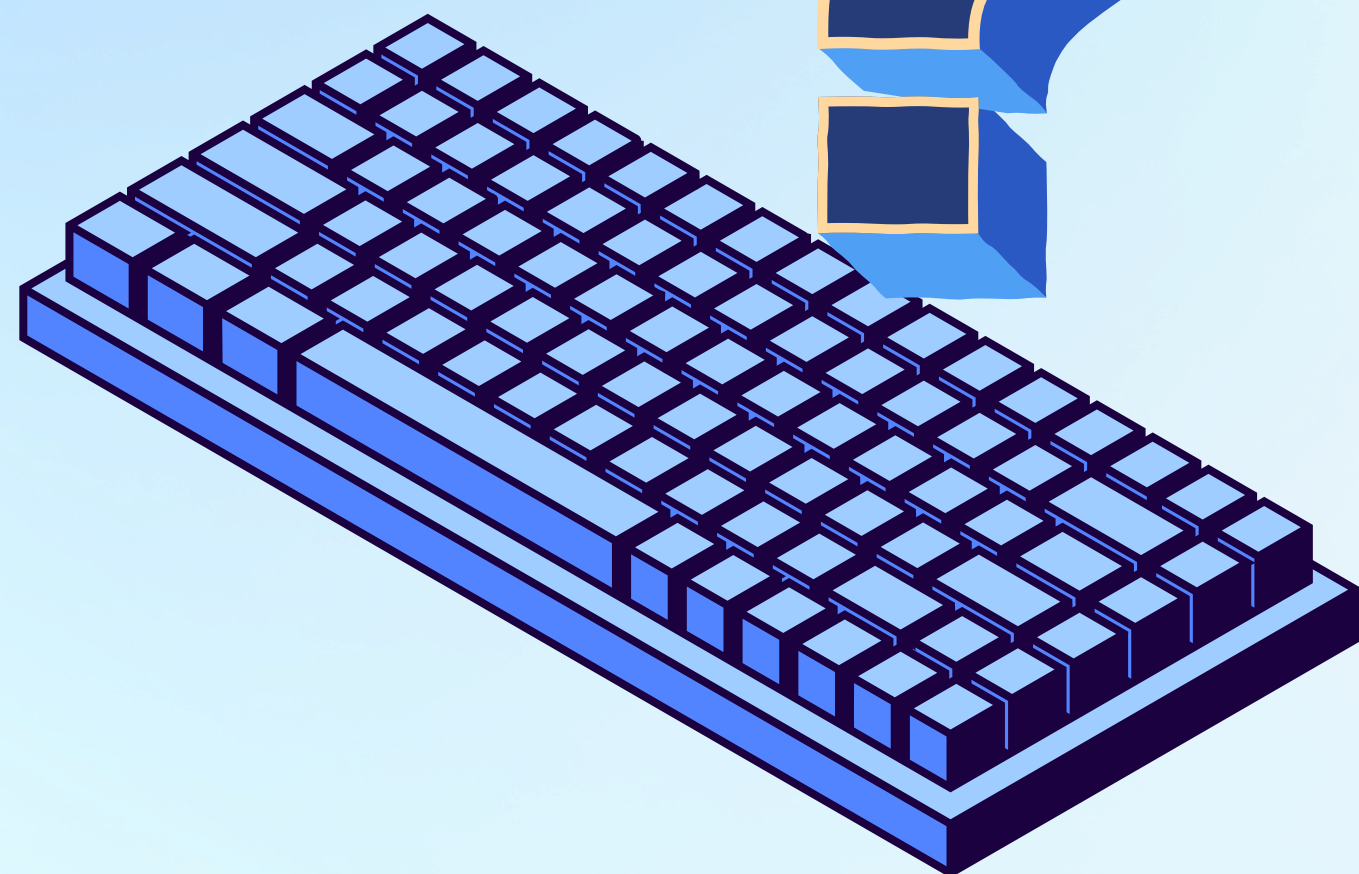
Constraints

- ห้าม Action ผิดลำดับ
- State ต้องถูกต้อง
- รองรับ Undo / Redo

Output

- Event Stack
- Redo Stack
- Current State





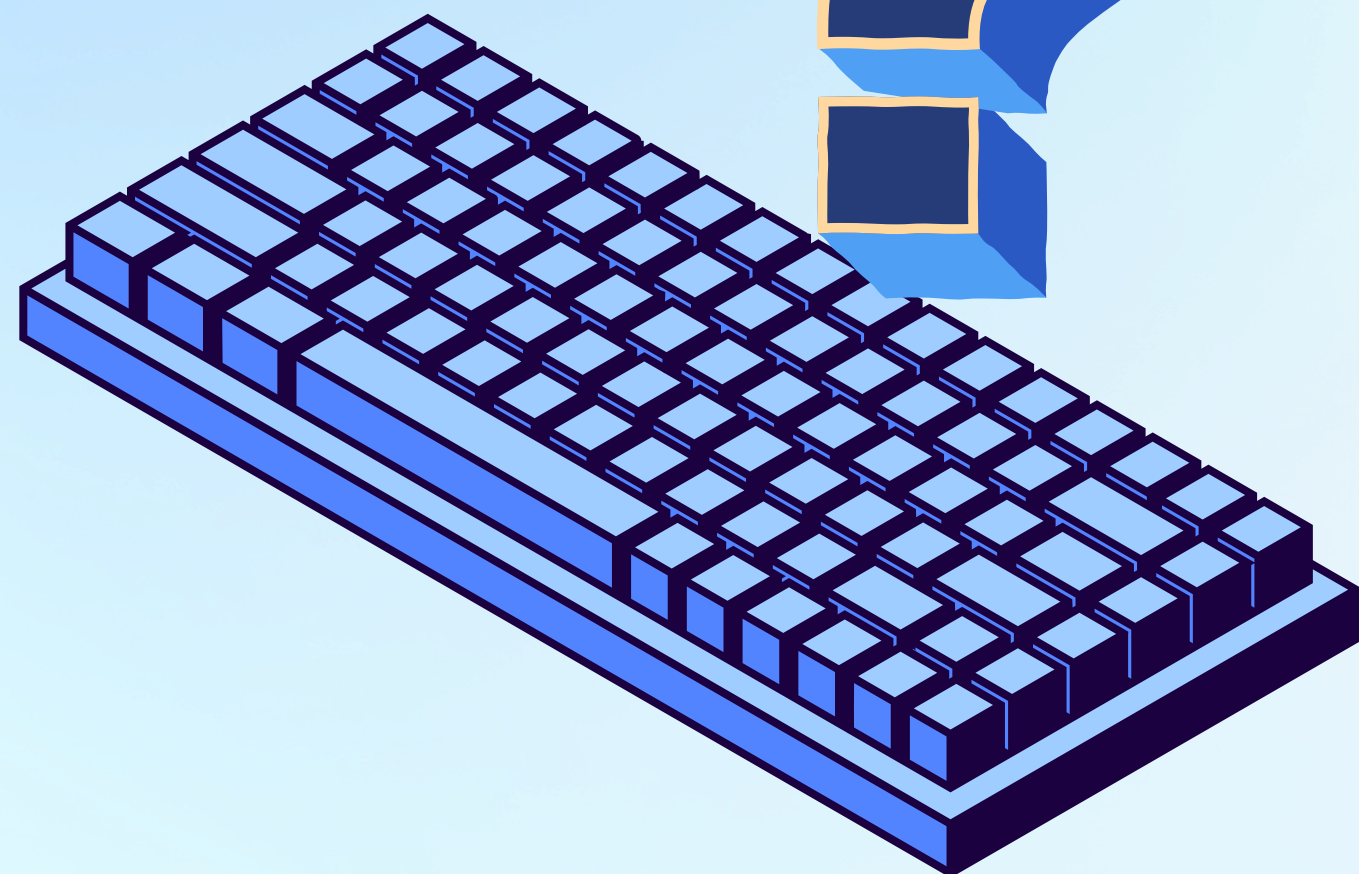
Algorithm A

Algorithm A: Event Stack

- ใช้โครงสร้างหลัก 3 ตัว
 - Event Stack
 - Transition Table
 - Redo Stack

1. รับ Action ใหม่
2. ตรวจสอบ Action กับ Transition Table 2.1 หาสถานะปัจจุบัน
 - 2.2 ตรวจสอบสถานะถัดไป
 - 2.3 ถ้า Transition ไม่ถูกต้อง ไม่เพิ่ม Action และจบการทำงาน
 - 2.4 ถ้า Transition ถูกต้อง Push Event เข้า Event Stack ล้าง Redo Stack
3. Undo
 - 3.1 Pop Event จาก Event Stack
 - 3.2 Push Event เข้า Redo Stack
4. Redo
 - 4.1 Pop Event จาก Redo Stack
 - 4.2 ตรวจสอบ Transition อีกครั้ง
 - 4.3 Push Event กลับเข้า Event Stack
5. ตรวจสอบและคืนค่า currentState





Algorithm B: Event Stack + State Machine

หลักการทำงาน

Algorithm B ทำงานคล้าย Algorithm A แต่มีตัวแปร

currentState

สำหรับเก็บสถานะปัจจุบันโดยตรง

Add Action

Action → Transition Table → Next State → Push Event
→ Update State

Undo

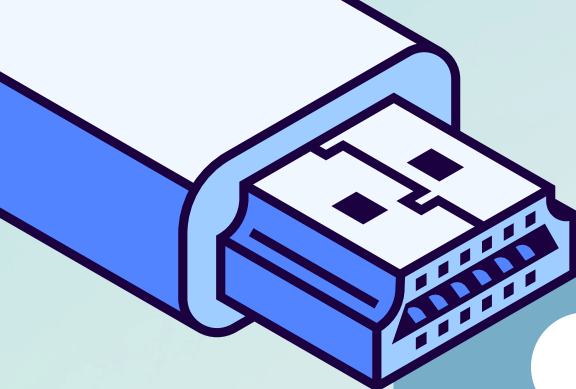
Pop Event → Push Redo → currentState = Previous State

Redo

Pop Redo → ตรวจสอบ Transition → Push Event
→ currentState = Next State

- ใช้โครงสร้างหลัก 3 ตัว
 - Event Stack
 - Transition Table
 - Redo Stack





Undo / Redo

Undo Event Stack

$A \rightarrow B \rightarrow C$

กด Undo

Event Stack

$A \rightarrow B$

Redo Stack

C

Redo

กด Redo

Event Stack

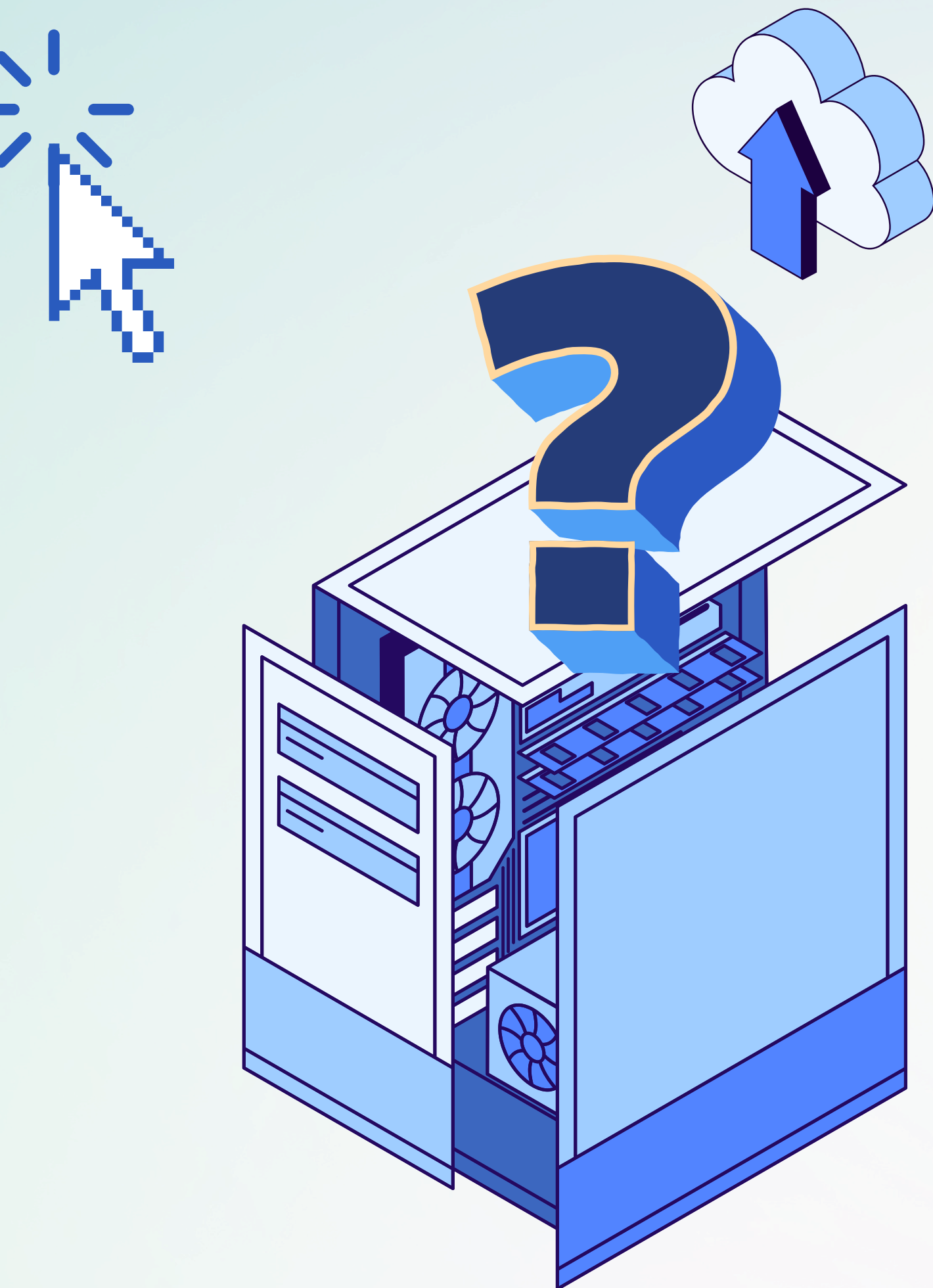
$A \rightarrow B \rightarrow C$

Redo Stack

ว่าง

ถ้ามีการเพิ่ม Action ใหม่หลังจาก Undo ระบบจะ ล้าง Redo Stack เพื่อไม่ให้เกิดประวัติที่ขัดแย้งกัน





เปรียบเทียบ Algorithm A และ B

หัวข้อ	Algorithm A	Algorithm B
Event Stack	✓	✓
Redo Stack	✓	✓
Transition Table	✓	✓
Undo	✓	✓
Redo	✓	✓
เก็บ Current State โดยตรง	✗	✓
State Machine	ใช้ตรวจสอบ Transition	ใช้ร่วมกับ Current State

สรุป

Algorithm A เน้นใช้ Event Stack เป็นหลัก Algorithm B เพิ่ม State Machine + currentState ทำให้สามารถติดตามสถานะปัจจุบันได้โดยตรง



Transition Table

Transition Table ทำหน้าที่ตรวจสอบว่า Action สามารถเกิดขึ้นใน State ปัจจุบันได้หรือไม่

Current State	Action	Next State
NEW	CALL_RECEIVED	RECEIVED
RECEIVED	TEAM_ASSIGNED	ASSIGNED
ASSIGNED	VEHICLE_DISPATCHED	DISPATCHED
DISPATCHED	ARRIVED_AT_SCENE	ON_SCENE
ON_SCENE	CASE_CLOSED	CLOSED

ถ้า Action ไม่ตรงกับ State ปัจจุบัน
→ nextState = null
→ ไม่สามารถเพิ่ม Action ได้



×

Flowchart

1. หน้าปก

↓

2. Case Story

↓

3. Input / Output / Constraints

↓

4. Transition Table

↓

5. Algorithm A: Event Stack

↓

6. Flowchart Algorithm A

↓

7. Algorithm B: Event Stack + State Machine

↓

8. Flowchart Algorithm B

↓

9. Test Cases

↓

10. Correctness Invariant

↓

11. คำถามวิเคราะห์ / สรุป

